Shape Analysis by Abstract Interpretation of Non-Blocking Concurrent Programs

(Ongoing Work)

Valentin Barbazo Xavier Rival

École Normale Supérieure – Université PSL

13th November 2025

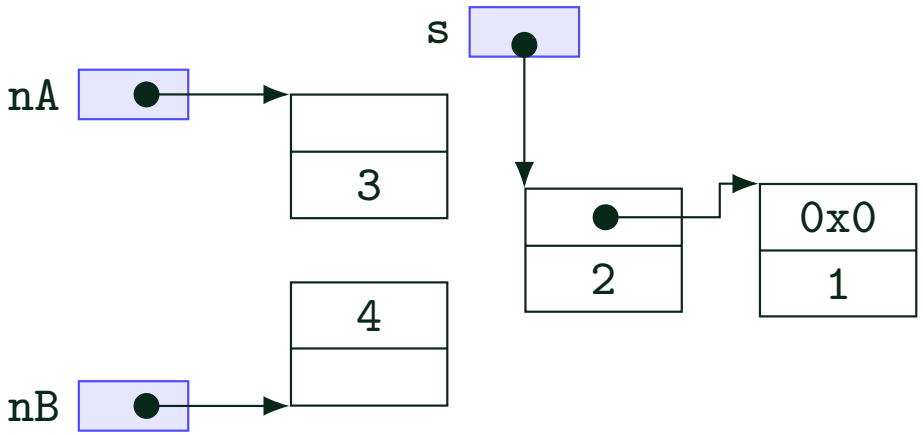






Initial Example

Two unsynchronized threads push on a shared stack



Thread A

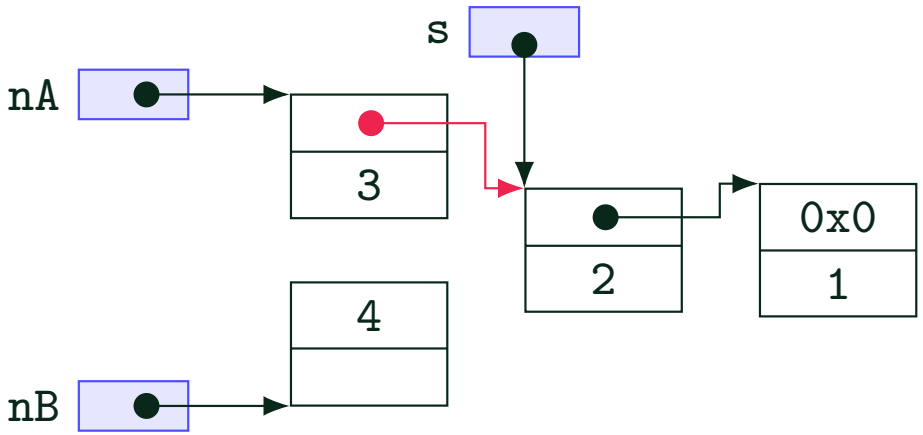
```
A1:   nA->next = s;
A2:   s = nA;
```

Thread B

```
B1:   nB->next = s;
B2:   s = nB;
```

Initial Example

Two unsynchronized threads push on a shared stack



Thread A

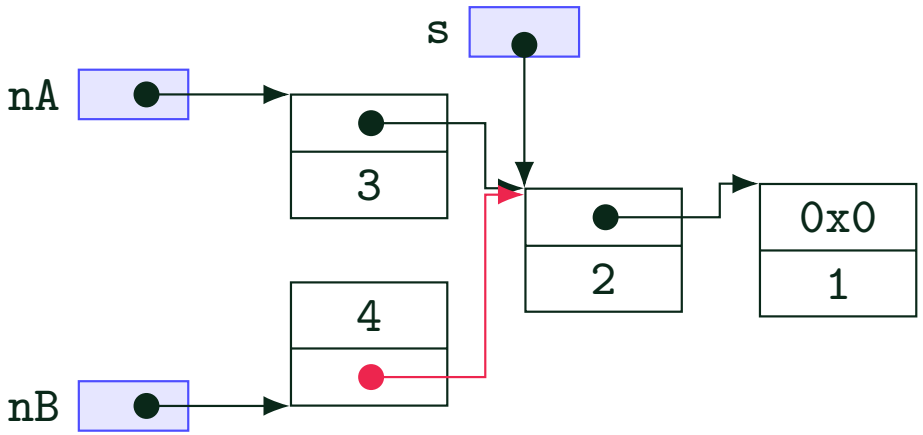
```
A1:   nA->next = s;
A2:   s = nA;
```

Thread B

```
B1:   nB->next = s;
B2:   s = nB;
```

Initial Example

Two unsynchronized threads push on a shared stack



Thread A

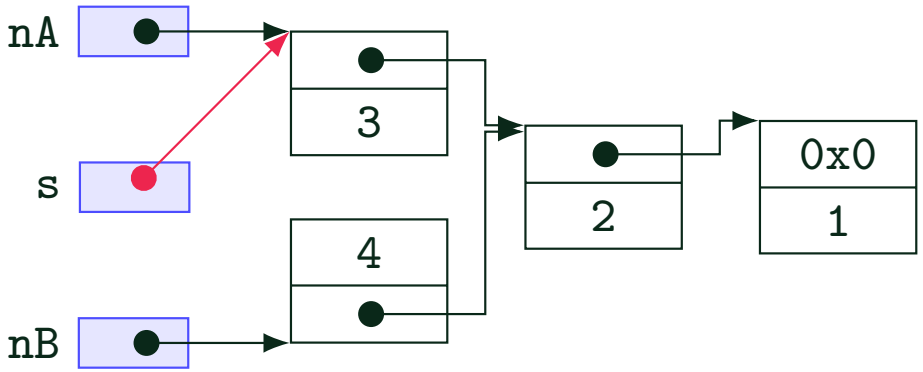
```
A1:   nA->next = s;
A2:   s = nA;
```

Thread B

```
B1:   nB->next = s;
B2:   s = nB;
```

Initial Example

Two unsynchronized threads push on a shared stack



Thread A

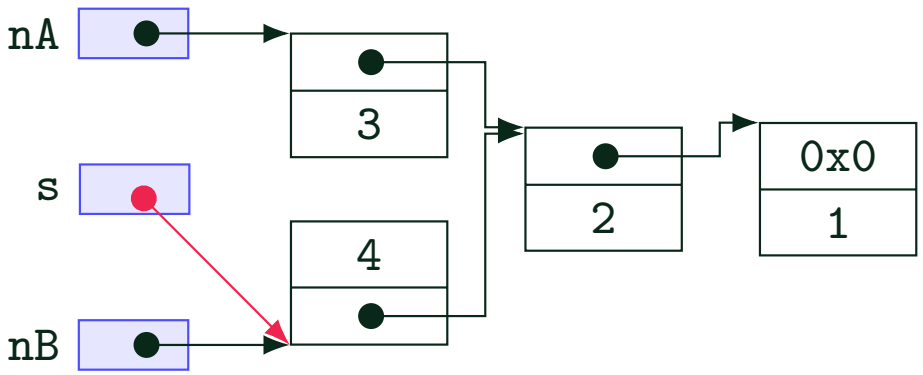
```
A1:   nA->next = s;
A2:   s = nA;
```

Thread B

```
B1:   nB->next = s;
B2:   s = nB;
```

Initial Example

Two unsynchronized threads push on a shared stack



Thread A

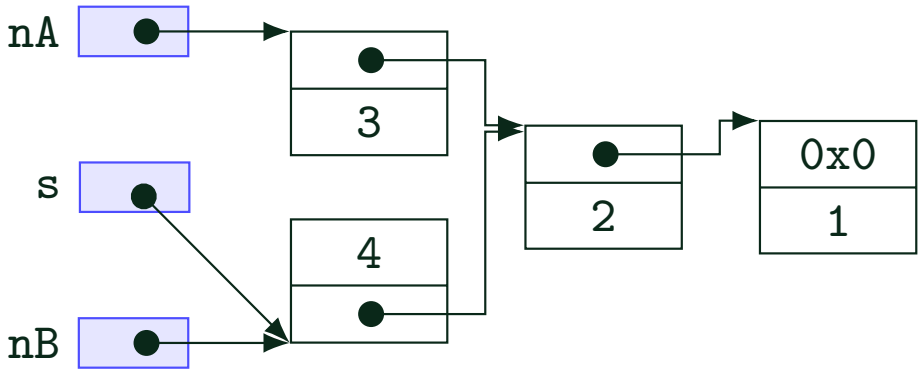
```
A1:   nA->next = s;
A2:   s = nA;
```

Thread B

```
B1:   nB->next = s;
B2:   s = nB;
```

Initial Example

Two unsynchronized threads push on a shared stack



Thread A

```
A1:   nA->next = s;
A2:   s = nA;
```

Thread B

```
B1:   nB->next = s;
B2:   s = nB;
```

- The linked-list structure starting from `s` is preserved ✓
- Thread B has the undesirable side effect of removing the node pointed to by `nA` from the structure ✗

Initial Example

(Non-)Blocking Synchronization

- Several solutions to the previous problem using lock-based synchronization
  $\rightarrow$ may introduce other problems (performance, deadlocks...)
- Non-blocking concurrency relies on wrapping shared-memory access in small uninterruptible[1] critical sections
  - Atomic memory read
  - Compare-and-Swap instruction

  $\Rightarrow$ We add an **atomic** keyword in our program syntax to model such critical sections

[1] Usually hardware-provided guarantee

Initial Example

(Non-)Blocking Synchronization

Our Compare-and-Swap instruction $\texttt{x} = \textbf{CAS}(e_0, e_1, e_2)$ is interpreted as the following critical section:

$$
\begin{array}{l}
\textbf{atomic(}\ \textbf{if}\ *e_0 = *e_1\ \textbf{then} \\
\qquad\qquad *e_0 = e_2; \\
\qquad\qquad \texttt{x} = \texttt{true}; \\
\qquad \textbf{else} \\
\qquad\qquad \texttt{x} = \texttt{false};\ \textbf{)}
\end{array}
$$

Initial Examplev

Solution using CAS

Thread A

```
A1: okA = false;
A2: while (!okA) {
A3:   atomic(tA = s);
A4:   nA->next = tA;
A5:   okA = CAS(&s,&tA,nA); }
```

Thread B

```
B1: okB = false;
B2: while (!okB) {
B3:   atomic(tB = s);
B4:   nB->next = tB;
B5:   okB = CAS(&s,&tB,nB); }
```

What do we want to prove on this program?

In any interleaving:

- The linked-list structure starting from s is preserved
- Both nodes belong to the list when execution ends

Initial Example

Challenges

Thread A

```
A1: okA = false;
A2: while (!okA) {
A3:   atomic(tA = s);
A4:   nA->next = tA;
A5:   okA = CAS(&s,&tA,nA); }
```

Thread B

```
B1: okB = false;
B2: while (!okB) {
B3:   atomic(tB = s);
B4:   nB->next = tB;
B5:   okB = CAS(&s,&tB,nB); }
```

Reasoning formally about this program is challenging:

Initial Example

Challenges

Thread A

```
A1: okA = false;
A2: while (!okA) {
A3:   atomic(tA = s);
A4:   nA->next = tA;
A5:   okA = CAS(&s,&tA,nA); }
```

Thread B

```
B1: okB = false;
B2: while (!okB) {
B3:   atomic(tB = s);
B4:   nB->next = tB;
B5:   okB = CAS(&s,&tB,nB); }
```

Reasoning formally about this program is challenging:

- it involves **additional state**

Initial Example

Challenges

Thread A

```
A1: okA = false;
A2: while (!okA) {
A3:   atomic(tA = s);
A4:   nA->next = tA;
A5:   okA = CAS(&s,&tA,nA); }
```

Thread B

```
B1: okB = false;
B2: while (!okB) {
B3:   atomic(tB = s);
B4:   nB->next = tB;
B5:   okB = CAS(&s,&tB,nB); }
```

Reasoning formally about this program is challenging:

- it involves **additional state** and **critical sections**

Initial Example

Challenges

Thread A

```
A1: okA = false;
A2: while (!okA) {
A3:   atomic(tA = s);
A4:   nA->next = tA;
A5:   okA = CAS(&s,&tA,nA); }
```

Thread B

```
B1: okB = false;
B2: while (!okB) {
B3:   atomic(tB = s);
B4:   nB->next = tB;
B5:   okB = CAS(&s,&tB,nB); }
```

Reasoning formally about this program is challenging:

- it involves **additional state** and **critical sections** in the presence of **loops**

Initial Example

Challenges

Thread A

```
A1: okA = false;
A2: while (!okA) {
A3:   atomic(tA = s);
A4:   nA->next = tA;
A5:   okA = CAS(&s,&tA,nA); }
```

Thread B

```
B1: okB = false;
B2: while (!okB) {
B3:   atomic(tB = s);
B4:   nB->next = tB;
B5:   okB = CAS(&s,&tB,nB); }
```

Reasoning formally about this program is challenging:

- it involves **additional state** and **critical sections** in the presence of **loops**
- the number of execution of each loop depends on the actual interleaving (hence of the other thread's behaviour)

Initial Example

Challenges

Thread A

```
A1: okA = false;
A2: while (!okA) {
A3:   atomic(tA = s);
A4:   nA->next = tA;
A5:   okA = CAS(&s,&tA,nA); }
```

Thread B

```
B1: okB = false;
B2: while (!okB) {
B3:   atomic(tB = s);
B4:   nB->next = tB;
B5:   okB = CAS(&s,&tB,nB); }
```

Reasoning formally about this program is challenging:

- it involves **additional state** and **critical sections** in the presence of **loops**
- the number of execution of each loop depends on the actual interleaving (hence of the other thread's behaviour)
- non-blocking concurrency does not restrict interleavings

Problem Statement

Infer the preservation of structural invariants in shared-memory multithreaded programs manipulating dynamic data structures:

- without enumerating all possible interleavings
- while still precisely account for pointer manipulations and thread interactions

Problem Statement

Infer the preservation of structural invariants in shared-memory multithreaded programs manipulating dynamic data structures:

- without enumerating all possible interleavings
- while still precisely account for pointer manipulations and thread interactions

Scope and Assumptions:

- Sequential consistency
- Non-blocking synchronization
- Fixed finite number of threads
- No memory deallocation

Rely-Guarantee in Abstract Interpretation

- Reachable local states: $\mathcal{R}(t) \rightarrow$ state assertions
- Interferences: $\mathcal{I}(t) \rightarrow$ state relations

Thread-modular interference semantics:

- Analyze each thread in **isolation**.
- During the analysis of a thread t:
  - use interferences $\mathcal{I}(t')$ from other threads $t' \neq t$ to update t's current state
  - enrich $\mathcal{I}(t)$ upon discovery of new shared memory transformations

$\Rightarrow$ constructive fixpoint computation of $\mathcal{R}$ and $\mathcal{I}$

Miné, 'Relational Thread-Modular Static Value Analysis by Abstract Interpretation', 2014 [4]

Rely-Guarantee in Abstract Interpretation

Connection with the RGSep parallel rule:

Example with two threads:

- Each thread is the environment for the other
- $\mathcal{I}(t_1) \approx G_1$ and $\mathcal{I}(t_2) \approx G_2$
- $\mathcal{R}(t_i)[\texttt{entry}] \approx P_i$ and $\mathcal{R}(t_i)[\texttt{exit}] \approx Q_i$ for $i \in \{1, 2\}$

$$\frac{\begin{array}{c} G_2, G_1 \vdash \{P_1\}\, C_1\, \{Q_1\} \\ G_1, G_2 \vdash \{P_2\}\, C_2\, \{Q_2\} \end{array}}{\emptyset, G_1 \cup G_2 \vdash \{P_1 * P_2\}\, C_1 \mid\mid C_2\, \{Q_1 * Q_2\}}\ \text{(RGSep-Par)}$$

Vafeiadis and Parkinson, 'A Marriage of Rely/Guarantee and Separation Logic', 2007 [5]

Rely-Guarantee in Abstract Interpretation

Connection with the RGSep parallel rule:

Example with two threads:

- Each thread is the environment for the other
- $\mathcal{I}(t_1) \approx G_1$ and $\mathcal{I}(t_2) \approx G_2$
- $\mathcal{R}(t_i)[\texttt{entry}] \approx P_i$ and $\mathcal{R}(t_i)[\texttt{exit}] \approx Q_i$ for $i \in \{1, 2\}$

$$\frac{\begin{array}{c} G_2, G_1 \vdash \{P_1\}\, C_1\, \{Q_1\} \\ G_1, G_2 \vdash \{P_2\}\, C_2\, \{Q_2\} \end{array}}{\emptyset, G_1 \cup G_2 \vdash \{P_1 * P_2\}\, C_1 \mid\mid C_2\, \{Q_1 * Q_2\}}\ \text{(RGSep-Par)}$$

We now need **suitable memory abstractions** to derive an effective static analysis!

Vafeiadis and Parkinson, 'A Marriage of Rely/Guarantee and Separation Logic', 2007 [5]

Memory State Abstraction

Separation-Logic based abstract domain to summarize memory states:

- **Symbolic variables** $(\alpha, \beta, \delta, \ldots)$ to abstract addresses and values
- **Predicates** over symbolic variables:

Chang and Rival, 'Relational inductive shape analysis', 2008 [1]

Memory State Abstraction

Separation-Logic based abstract domain to summarize memory states:

- **Symbolic variables** $(\alpha, \beta, \delta, \ldots)$ to abstract addresses and values
- **Predicates** over symbolic variables:
  - points-to predicate for a single memory cell: $\alpha \cdot \mathtt{f} \mapsto \beta$

Chang and Rival, 'Relational inductive shape analysis', 2008 [1]

Memory State Abstraction

Separation-Logic based abstract domain to summarize memory states:

- **Symbolic variables** $(\alpha, \beta, \delta, \ldots)$ to abstract addresses and values
- **Predicates** over symbolic variables:
  - points-to predicate for a single memory cell: $\alpha \cdot \mathtt{f} \mapsto \beta$
  - inductive/segment predicate for unbounded complete/partial dynamic data structures: $\alpha \cdot \mathbf{list}$ and $\alpha \cdot \mathbf{lseg}$ for singly-linked lists

Chang and Rival, 'Relational inductive shape analysis', 2008 [1]

Memory State Abstraction

Separation-Logic based abstract domain to summarize memory states:

- **Symbolic variables** $(\alpha, \beta, \delta, \ldots)$ to abstract addresses and values
- **Predicates** over symbolic variables:
  - points-to predicate for a single memory cell: $\alpha \cdot \mathtt{f} \mapsto \beta$
  - inductive/segment predicate for unbounded complete/partial dynamic data structures: $\alpha \cdot \mathbf{list}$ and $\alpha \cdot \mathbf{lseg}$ for singly-linked lists
  - separating conjunction $*$ to combine predicates

Chang and Rival, 'Relational inductive shape analysis', 2008 [1]

Memory State Abstraction

Separation-Logic based abstract domain to summarize memory states:

- **Symbolic variables** $(\alpha, \beta, \delta, \ldots)$ to abstract addresses and values
- **Predicates** over symbolic variables:
  - points-to predicate for a single memory cell: $\alpha \cdot \mathtt{f} \mapsto \beta$
  - inductive/segment predicate for unbounded complete/partial dynamic data structures: $\alpha \cdot \textbf{list}$ and $\alpha \cdot \textbf{lseg}$ for singly-linked lists
  - separating conjunction $*$ to combine predicates
- **Concretization**: $\gamma_{\mathbb{M}^\sharp} : \mathbb{M}^\sharp \longrightarrow \mathcal{P}(\mathbb{M})$

Chang and Rival, 'Relational inductive shape analysis', 2008 [1]

Memory State Abstraction

Separation-Logic based abstract domain to summarize memory states:

- **Symbolic variables** $(\alpha, \beta, \delta, \ldots)$ to abstract addresses and values
- **Predicates** over symbolic variables:
  - points-to predicate for a single memory cell: $\alpha \cdot \mathtt{f} \mapsto \beta$
  - inductive/segment predicate for unbounded complete/partial dynamic data structures: $\alpha \cdot$ **list** and $\alpha \cdot$ **lseg** for singly-linked lists
  - separating conjunction $*$ to combine predicates
- **Concretization**: $\gamma_{\mathbb{M}^\sharp} : \mathbb{M}^\sharp \longrightarrow \mathcal{P}(\mathbb{M})$
- **Shape graph** representation

Chang and Rival, 'Relational inductive shape analysis', 2008 [1]

Memory State Abstraction

Separation-Logic based abstract domain to summarize memory states:

- **Symbolic variables** $(\alpha, \beta, \delta, \ldots)$ to abstract addresses and values
- **Predicates** over symbolic variables:
  - points-to predicate for a single memory cell: $\alpha \cdot \mathtt{f} \mapsto \beta$
  - inductive/segment predicate for unbounded complete/partial dynamic data structures: $\alpha \cdot$ **list** and $\alpha \cdot$ **lseg** for singly-linked lists
  - separating conjunction $*$ to combine predicates
- **Concretization**: $\gamma_{\mathbb{M}^\sharp} : \mathbb{M}^\sharp \longrightarrow \mathcal{P}(\mathbb{M})$
- **Shape graph** representation

Extended with some **may-shared** information to denote memory that may be accessible to other threads.
$\Rightarrow$ An abstract state represents the point of view of a single thread on the memory.

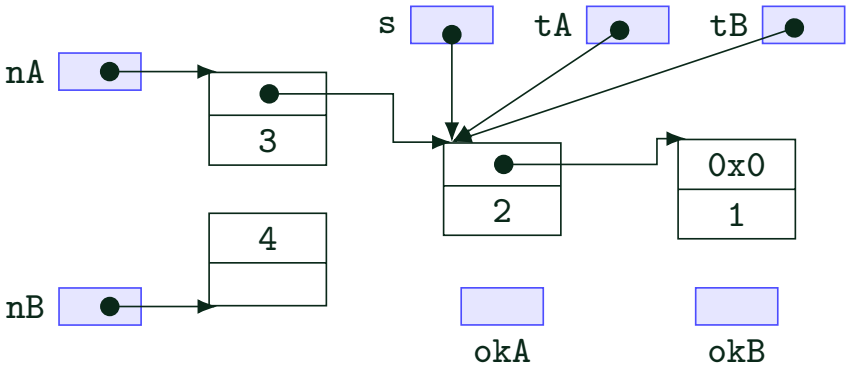
Chang and Rival, 'Relational inductive shape analysis', 2008 [1]

Memory State Abstraction

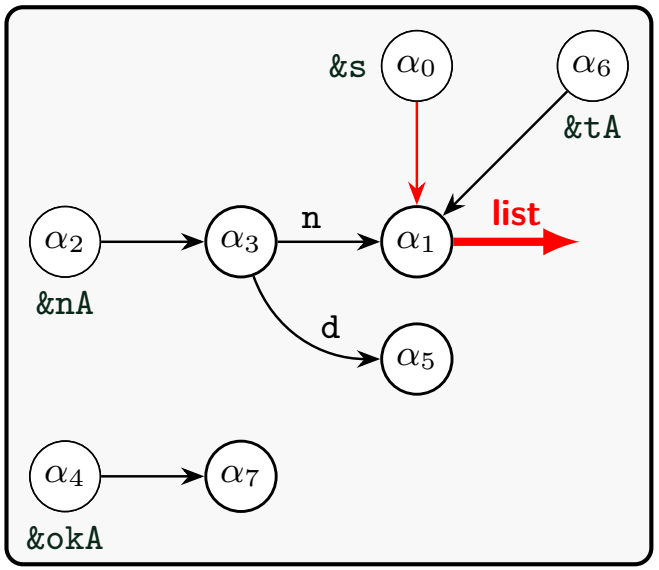
Example

Abstracting a state from our initial example from the point of view of Thread A:

- only keep the shared state and Thread A's local state
- may-shared information is displayed using red predicates



$m_A^\sharp =$



⚠ Concrete memory cells are **abstracted by arrows** in the shape graph, not nodes!

Relational Memory Abstraction

Relational extension of the shape abstraction to summarize memory transformations:

- Identical shape predicates $m^\sharp$ to represent memory
- **Relational connectives** over shape predicates:

Illous *et al.*, 'A relational shape abstract domain', 2021 [3]

Relational Memory Abstraction

Relational extension of the shape abstraction to summarize memory transformations:

- Identical shape predicates $m^\sharp$ to represent memory
- **Relational connectives** over shape predicates:
  - Identity connective: $\texttt{Id}(m^\sharp)$
    describes pairs of memories that are equal and both abstracted by $m^\sharp$

Illous *et al.*, 'A relational shape abstract domain', 2021 [3]

Relational Memory Abstraction

Relational extension of the shape abstraction to summarize memory transformations:

- Identical shape predicates $m^\sharp$ to represent memory
- **Relational connectives** over shape predicates:
  - Identity connective: $\texttt{Id}(m^\sharp)$
    describes pairs of memories that are equal and both abstracted by $m^\sharp$
  - Transform-into connective: $[m_i^\sharp \dashrightarrow m_o^\sharp]$
    describes pairs corresponding to the transformation of a memory abstracted by $m_i^\sharp$ into a memory abstracted by $m_o^\sharp$

Illous *et al.*, 'A relational shape abstract domain', 2021 [3]

Relational Memory Abstraction

Relational extension of the shape abstraction to summarize memory transformations:

- Identical shape predicates $m^\sharp$ to represent memory
- **Relational connectives** over shape predicates:
  - Identity connective: $\texttt{Id}(m^\sharp)$
    describes pairs of memories that are equal and both abstracted by $m^\sharp$
  - Transform-into connective: $[m_i^\sharp \dashrightarrow m_o^\sharp]$
    describes pairs corresponding to the transformation of a memory abstracted by $m_i^\sharp$ into a memory abstracted by $m_o^\sharp$
  - Relational separating conjunction $r_1^\sharp *_{\mathtt{R}} r_2^\sharp$
    to combine connectives describing transformations on disjoint memory regions

Illous *et al.*, 'A relational shape abstract domain', 2021 [3]

Relational Memory Abstraction

Relational extension of the shape abstraction to summarize memory transformations:

- Identical shape predicates $m^\sharp$ to represent memory
- **Relational connectives** over shape predicates:
  - Identity connective: $\texttt{Id}(m^\sharp)$
    describes pairs of memories that are equal and both abstracted by $m^\sharp$
  - Transform-into connective: $[m_i^\sharp \dashrightarrow m_o^\sharp]$
    describes pairs corresponding to the transformation of a memory abstracted by $m_i^\sharp$ into a memory abstracted by $m_o^\sharp$
  - Relational separating conjunction $r_1^\sharp *_{\mathtt{R}} r_2^\sharp$
    to combine connectives describing transformations on disjoint memory regions
- **Concretization**: $\gamma_{\mathbb{R}^\sharp} : \mathbb{R}^\sharp \longrightarrow \mathcal{P}(\mathbb{M} \times \mathbb{M})$

Illous *et al.*, 'A relational shape abstract domain', 2021 [3]

Relational Memory Abstraction

Relational extension of the shape abstraction to summarize memory transformations:

- Identical shape predicates $m^\sharp$ to represent memory
- **Relational connectives** over shape predicates:
  - Identity connective: $\texttt{Id}(m^\sharp)$
    describes pairs of memories that are equal and both abstracted by $m^\sharp$
  - Transform-into connective: $[m_i^\sharp \dashrightarrow m_o^\sharp]$
    describes pairs corresponding to the transformation of a memory abstracted by $m_i^\sharp$ into a memory abstracted by $m_o^\sharp$
  - Relational separating conjunction $r_1^\sharp *_{\mathtt{R}} r_2^\sharp$
    to combine connectives describing transformations on disjoint memory regions
- **Concretization**: $\gamma_{\mathbb{R}^\sharp} : \mathbb{R}^\sharp \longrightarrow \mathcal{P}(\mathbb{M} \times \mathbb{M})$

- Abstract **input-output** relational semantics of programs
- Design a compositional **interprocedural** shape analysis

Illous *et al.*, 'A relational shape abstract domain', 2021 [3]

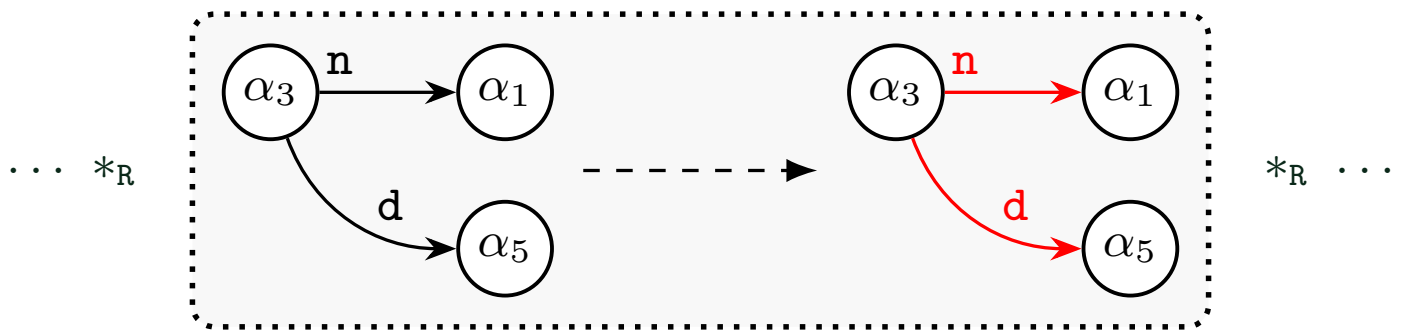
Relational Memory Abstraction

Describing ownership transfer

During execution of Thread A:

- Before CAS: the memory cell accessible from `nA` is in the local state
- After CAS succeeds: the shared variable `s` points to the same memory cell as `nA` making it accessible to other threads

This **transfer of ownership** can be represented with the following abstract relation:



Combining Abstractions

Our analysis works as follows:

- **Thread-modular** Rely-Guarantee analysis based on previous work

Combining Abstractions

Our analysis works as follows:

- **Thread-modular** Rely-Guarantee analysis based on previous work
- **Combine the state and relational analyses**:
  - Relational analysis on **critical sections only**
    $\rightarrow$ direct relational representation of **shared state interference**
  - State analysis for all **other statements**
    $\rightarrow$ we require these instructions to **access local predicates only**

Combining Abstractions

Our analysis works as follows:

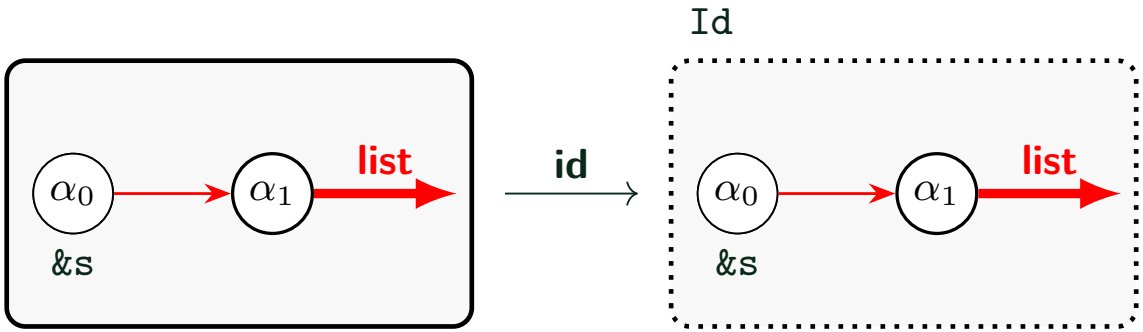
- **Thread-modular** Rely-Guarantee analysis based on previous work
- **Combine the state and relational analyses**:
  - Relational analysis on **critical sections only**
    $\rightarrow$ direct relational representation of **shared state interference**
  - State analysis for all **other statements**
    $\rightarrow$ we require these instructions to **access local predicates only**
- **New operators** to make these abstractions interact:
  - From states to relations:
    id : $\mathbb{M}^\sharp \longrightarrow \mathbb{R}^\sharp$
  - From relations to states:
    output : $\mathbb{R}^\sharp \longrightarrow \mathbb{M}^\sharp$
  - Update a state according to the transformations described by an interference:
    apply : $\mathbb{R}^\sharp \times \mathbb{M}^\sharp \longrightarrow \mathcal{P}_{\text{fin}}\left(\mathbb{M}^\sharp\right)$

Operators

From states to relations

id : $\mathbb{M}^\sharp \longrightarrow \mathbb{R}^\sharp$

Transforms an abstract shape into the relational identity connective of the same shape.



Soundness:

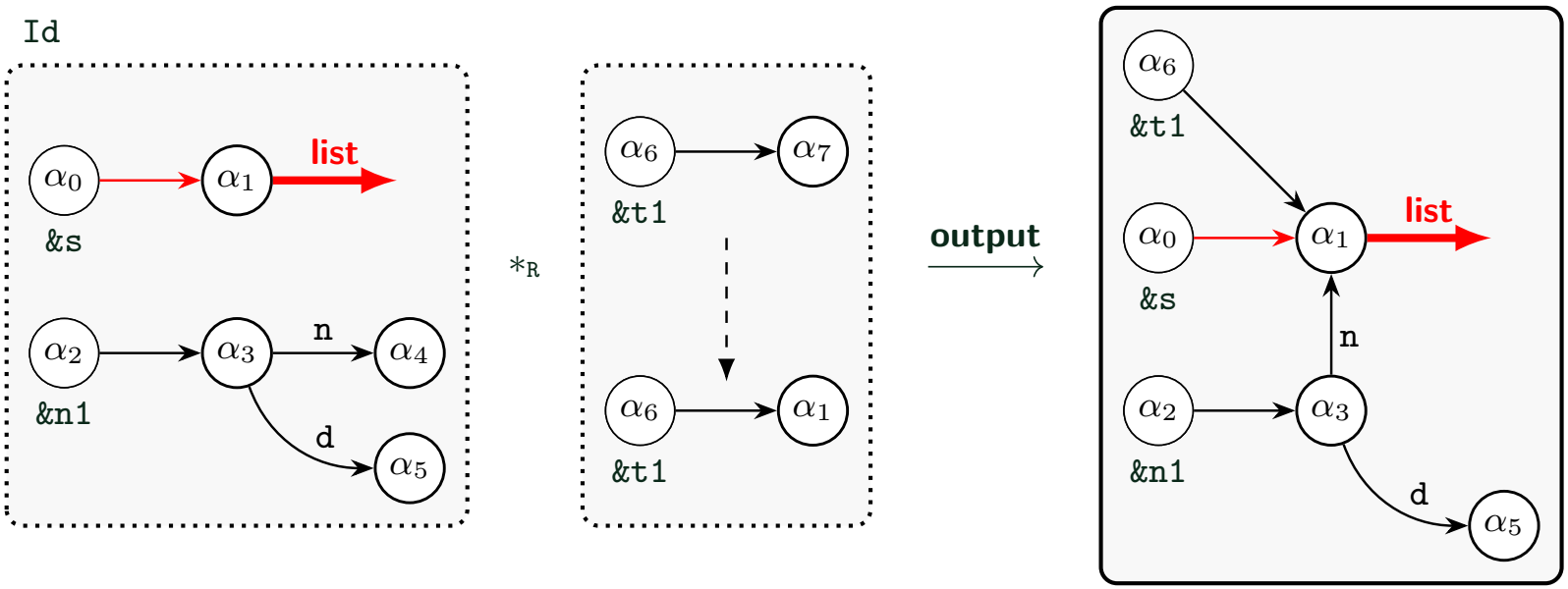
$$\forall m \in \mathbb{M} : m \in \gamma_{\mathbb{M}^\sharp}(m^\sharp) \implies (m, m) \in \gamma_{\mathbb{R}^\sharp}(\mathbf{id}(m^\sharp))$$

Operators

From relations to states

output : $\mathbb{R}^\sharp \longrightarrow \mathbb{M}^\sharp$

Produces an abstract state overapproximating all possible states at the end of a transformation described by an abstract relation.



Soundness:

$$\forall m_i, m_o \in \mathbb{M} : (m_i, m_o) \in \gamma_{\mathbb{R}^\sharp}(r^\sharp) \implies m_o \in \gamma_{\mathbb{M}^\sharp}(\textbf{output}(r^\sharp))$$

Operators

Apply a relation to a state

apply : $\mathbb{R}^\sharp \times \mathbb{M}^\sharp \longrightarrow \mathcal{P}_{\text{fin}}\left(\mathbb{M}^\sharp\right)$
From an abstract relation and an abstract state, compute a new state according to the transformations described by the relation.

- It amounts to **seeing critical regions as procedures** whose effects are summarized by an interference.
- Then **applying an interference is similar to an abstract procedure call** in an interprocedural analysis.

It relies on an *abstract composition* operator[1]:

$$\textbf{apply}(r^\sharp, m^\sharp) = \{\textbf{output}(m^{\sharp\prime}) \mid m^{\sharp\prime} \in M^\sharp \text{ and } M^\sharp = r^\sharp \circ^\sharp \textbf{id}(m^\sharp)\}$$

Soundness: If $\textbf{apply}(r^\sharp, m^\sharp) = \{m_1^\sharp, \ldots, m_n^\sharp\}$ then $\forall m_1, m_2 \in \mathbb{M}$:

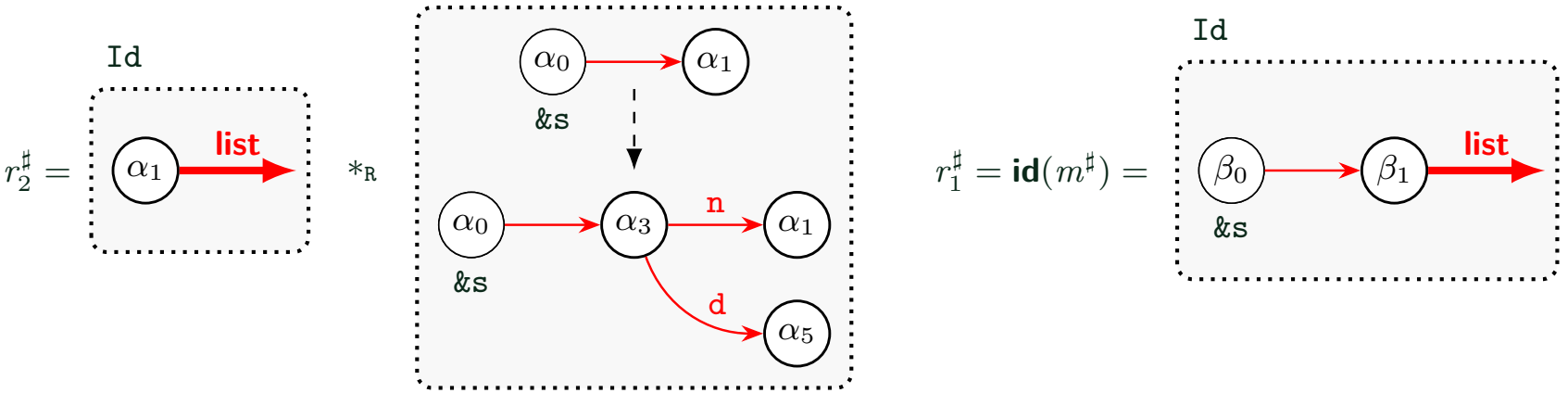
$$m_1 \in \gamma_{\mathbb{M}^\sharp}(m^\sharp) \wedge (m_1, m_2) \in \gamma_{\mathbb{R}^\sharp}(r^\sharp) \implies \exists i : m_2 \in \gamma_{\mathbb{M}^\sharp}(m_i^\sharp).$$

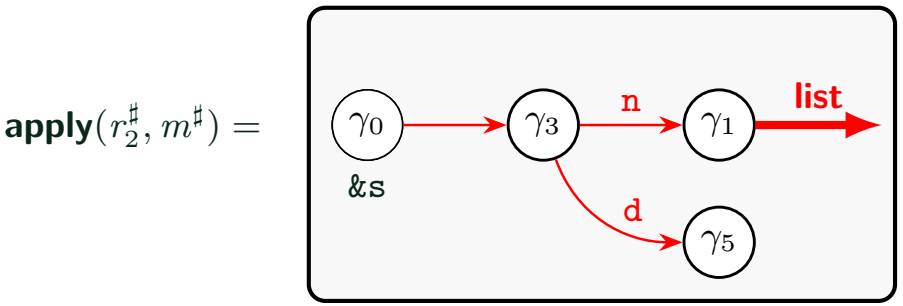
[1] Illous, 'Abstractions relationnelles de la mémoire pour une analyse compositionnelle de structures de données', 2019 [2]

Operators

Apply a relation to a state

When computing $r_2^\sharp \circ^\sharp r_1^\sharp$ we need to match the **output** parts of $r_1^\sharp$ with the **input** parts of $r_2^\sharp$.





Conclusion

What remains to do:

- Some elements of formalization and implementation
- Evaluation on a set of lock-free concurrent programs

Conclusion

What remains to do:

- Some elements of formalization and implementation
- Evaluation on a set of lock-free concurrent programs

Back to our assumptions:

- Handle unbounded number of threads with the notion of stabilization?
- Turn to the analysis of the *most general client* of a concurrent data structure?

Conclusion

What remains to do:

- Some elements of formalization and implementation
- Evaluation on a set of lock-free concurrent programs

Back to our assumptions:

- Handle unbounded number of threads with the notion of stabilization?
- Turn to the analysis of the *most general client* of a concurrent data structure?

Thank you!

References

[1] Bor-Yuh Evan Chang and Xavier Rival. 'Relational inductive shape analysis'. In: *Proceedings of the 35th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. POPL '08. San Francisco, California, USA: Association for Computing Machinery, 2008, pp. 247–260. DOI: 10.1145/1328438.1328469.

[2] Hugo Illous. 'Abstractions relationnelles de la mémoire pour une analyse compositionnelle de structures de données'. Theses. Université Paris sciences et lettres, Apr. 2019.

[3] Hugo Illous, Matthieu Lemerre and Xavier Rival. 'A relational shape abstract domain'. In: *Formal Methods in System Design* 57.3 (Sept. 2021), pp. 343–400. DOI: 10.1007/s10703-021-00366-4.

[4] Antoine Miné. 'Relational Thread-Modular Static Value Analysis by Abstract Interpretation'. In: *Verification, Model Checking, and Abstract Interpretation*. Ed. by Kenneth L. McMillan and Xavier Rival. Berlin, Heidelberg: Springer Berlin Heidelberg, 2014, pp. 39–58. DOI: 10.1007/978-3-642-54013-4_3.

[5] Viktor Vafeiadis and Matthew Parkinson. 'A Marriage of Rely/Guarantee and Separation Logic'. In: *CONCUR 2007 – Concurrency Theory*. Ed. by Luís Caires and Vasco T. Vasconcelos. Berlin, Heidelberg: Springer Berlin Heidelberg, 2007, pp. 256–271. DOI: 10.1007/978-3-540-74407-8_18.

Analysis Example

Thread A

```
A1: okA = false;
A2: while (!okA) {
A3:   atomic(tA = s);
A4:   nA->next = tA;
A5:   okA = CAS(&s,&tA,nA); }
```

Analysis Example

Thread A

```
A1: okA = false;
A2: while (!okA) {
A3:   atomic(tA = s);
A4:   nA->next = tA;
A5:   okA = CAS(&s,&tA,nA); }
```

- start with some initial abstract state $m^\sharp_{A1} \stackrel{\text{def}}{=} m^\sharp_{init}$

Analysis Example

Thread A

```
A1: okA = false;
A2: while (!okA) {
A3:   atomic(tA = s);
A4:   nA->next = tA;
A5:   okA = CAS(&s,&tA,nA); }
```

- start with some initial abstract state $m^\sharp_{A1} \stackrel{\text{def}}{=} m^\sharp_{init}$
- Upon reaching A3 with state $m^\sharp_{A3}$: start a relational analysis of the critical section with relation $r^\sharp_{A3} \stackrel{\text{def}}{=} \mathbf{id}(m^\sharp_{A3})$

Analysis Example

Thread A

```
A1: okA = false;
A2: while (!okA) {
A3:   atomic(tA = s);
A4:   nA->next = tA;
A5:   okA = CAS(&s,&tA,nA); }
```

- start with some initial abstract state $m^\sharp_{A1} \stackrel{\text{def}}{=} m^\sharp_{init}$
- Upon reaching A3 with state $m^\sharp_{A3}$: start a relational analysis of the critical section with relation $r^\sharp_{A3} \stackrel{\text{def}}{=} \mathbf{id}(m^\sharp_{A3})$
- Upon reaching the end of the critical section in A4 with relation $r^\sharp_{A4}$:

Analysis Example

Thread A

```
A1: okA = false;
A2: while (!okA) {
A3:   atomic(tA = s);
A4:   nA->next = tA;
A5:   okA = CAS(&s,&tA,nA); }
```

- start with some initial abstract state $m^\sharp_{A1} \stackrel{\text{def}}{=} m^\sharp_{init}$
- Upon reaching A3 with state $m^\sharp_{A3}$: start a relational analysis of the critical section with relation $r^\sharp_{A3} \stackrel{\text{def}}{=} \mathbf{id}(m^\sharp_{A3})$
- Upon reaching the end of the critical section in A4 with relation $r^\sharp_{A4}$:
  - Enrich Thread A's interferences with $f^\sharp(r^\sharp_{A4})$

Analysis Example

Thread A

```
A1: okA = false;
A2: while (!okA) {
A3:   atomic(tA = s);
A4:   nA->next = tA;
A5:   okA = CAS(&s,&tA,nA); }
```

- start with some initial abstract state $m^\sharp_{A1} \stackrel{\text{def}}{=} m^\sharp_{init}$
- Upon reaching A3 with state $m^\sharp_{A3}$: start a relational analysis of the critical section with relation $r^\sharp_{A3} \stackrel{\text{def}}{=} \mathbf{id}(m^\sharp_{A3})$
- Upon reaching the end of the critical section in A4 with relation $r^\sharp_{A4}$:
  - Enrich Thread A's interferences with $f^\sharp(r^\sharp_{A4})$
  - Get the output state $m^\sharp_{A4} \stackrel{\text{def}}{=} \mathbf{output}(r^\sharp_{A4})$

Analysis Example

Thread A

```
A1: okA = false;
A2: while (!okA) {
A3:   atomic(tA = s);
A4:   nA->next = tA;
A5:   okA = CAS(&s,&tA,nA); }
```

- start with some initial abstract state $m^\sharp_{A1} \stackrel{\text{def}}{=} m^\sharp_{init}$
- Upon reaching A3 with state $m^\sharp_{A3}$: start a relational analysis of the critical section with relation $r^\sharp_{A3} \stackrel{\text{def}}{=} \mathbf{id}(m^\sharp_{A3})$
- Upon reaching the end of the critical section in A4 with relation $r^\sharp_{A4}$:
  - Enrich Thread A's interferences with $f^\sharp(r^\sharp_{A4})$
  - Get the output state $m^\sharp_{A4} \stackrel{\text{def}}{=} \mathbf{output}(r^\sharp_{A4})$
  - Compute $m^{\sharp\prime}_{A4}$ by applying Thread B's known interferences to $m^\sharp_{A4}$ and continue the analysis with $m^{\sharp\prime}_{A4}$ until the next critical region in A5

Rely-Guarantee Separation Logic (RGSep)

State assertions (P, Q):

- SL assertions extended with **shared state assertions**: $\boxed{S}$
- The separating conjunction does not split the shared state:

$$\boxed{S_1} * \boxed{S_2} \Leftrightarrow \boxed{S_1 \wedge S_2}$$

Vafeiadis and Parkinson, 'A Marriage of Rely/Guarantee and Separation Logic', 2007 [5]

Rely-Guarantee Separation Logic (RGSep)

State assertions (P, Q):

- SL assertions extended with **shared state assertions**: $\boxed{S}$
- The separating conjunction does not split the shared state:

$$\boxed{S_1} * \boxed{S_2} \Leftrightarrow \boxed{S_1 \wedge S_2}$$

Rely and Guarantee relations (R, G):

- Sets of **actions** $S \rightsquigarrow S'$ representing interferences as shared state transformations (S and S' are regular SL assertions here)

Vafeiadis and Parkinson, 'A Marriage of Rely/Guarantee and Separation Logic', 2007 [5]

Rely-Guarantee Separation Logic (RGSep)

State assertions (P, Q):

- SL assertions extended with **shared state assertions**: $\boxed{S}$
- The separating conjunction does not split the shared state:

$$\boxed{S_1} * \boxed{S_2} \Leftrightarrow \boxed{S_1 \wedge S_2}$$

Rely and Guarantee relations (R, G):

- Sets of **actions** $S \rightsquigarrow S'$ representing interferences as shared state transformations (S and S' are regular SL assertions here)

$$\frac{R \cup G_2,\, G_1 \vdash \{P_1\}\, C_1\, \{Q_1\} \qquad R \cup G_1,\, G_2 \vdash \{P_2\}\, C_2\, \{Q_2\}}{R,\, G_1 \cup G_2 \vdash \{P_1 * P_2\}\, C_1 \parallel C_2\, \{Q_1 * Q_2\}}\ \text{(RGSep-Par)}$$

Vafeiadis and Parkinson, 'A Marriage of Rely/Guarantee and Separation Logic', 2007 [5]

Rely-Guarantee Separation Logic (RGSep)

State assertions (P, Q):

- SL assertions extended with **shared state assertions**: $\boxed{S}$
- The separating conjunction does not split the shared state:

$$\boxed{S_1} * \boxed{S_2} \Leftrightarrow \boxed{S_1 \wedge S_2}$$

Rely and Guarantee relations (R, G):

- Sets of **actions** $S \rightsquigarrow S'$ representing interferences as shared state transformations (S and S' are regular SL assertions here)

$$\frac{R \cup G_2,\, G_1 \vdash \{L_1 * \boxed{S_1}\}\, C_1\, \{L_1' * \boxed{S_1'}\} \qquad R \cup G_1,\, G_2 \vdash \{L_2 * \boxed{S_2}\}\, C_2\, \{L_2' * \boxed{S_2'}\}}{R,\, G_1 \cup G_2 \vdash \{L_1 * L_2 * \boxed{S_1 \wedge S_2}\}\, C_1 \parallel C_2\, \{L_1' * L_2' * \boxed{S_1' \wedge S_2'}\}}$$

Vafeiadis and Parkinson, 'A Marriage of Rely/Guarantee and Separation Logic', 2007 [5]